



DOUGLAS COLLEGE
COMMERCE AND BUSINESS
ADMINISTRATION

SafeSight

A Full-Stack Predictive Workplace Safety Management

System with reference to

Fairmont Waterfront



CSIS 4495 002: APPLIED RESEARCH PROJECT

PROGRESS REPORT 3

Name – Sandali Silva

Student number – 300394102

Table of Contents

1.	Introduction.....	3
1.1	Brief overview about the project.....	3
2.	Summary Description of Work Completed.....	3
2.1	Technical details of tasks performed.....	5
3.	Methodology	7
3.1	Issues encountered and how they were resolved	7
3.2	Changes made to the original project proposal.....	8
3.4	Updated timeline.....	9
3.5	Next Steps	10
4	Repository Check-In Summary.....	11
	Work Log	13
	AI Use Disclosure.....	14

1. Introduction

1.1 Brief overview about the project

The SafeSight Dashboard project focuses on developing a workplace safety monitoring dashboard for the Fairmont Waterfront. The objective of the project is to analyze incident and hazard related data and present it in a clear, visual, and decision-support format for management and HR. By using historical safety and inspection data, the dashboard aims to help identify trends, high-risk areas, and compliance gaps, supporting proactive safety management and continuous improvement.

2. Summary Description of Work Completed

Overview of work completed during this reporting period

During this reporting period, the project advanced into the final stages of Compliance Automation and System Integration. The focus shifted from simple data visualization to programmatic interaction with official regulatory documents. Key milestones achieved include:

1. Development of the Compliance Automation Engine (Objective 4)

A PDF generation service was developed using the pdf-lib library. This engine functions as a bridge between the MySQL database and the official WorkSafeBC EIIR (Employer Incident Investigation Report) templates. This development represents a transition from basic data storage to automated regulatory reporting, supporting the project objective of reducing administrative reporting time by 50%.

2. Engineering of the XFA PDF Mapping Logic

The system was enhanced to support complex XFA (XML Forms Architecture) PDF structures commonly used by government agencies. A deep-mapping strategy was implemented to bypass standard PDF encryption and enable the programmatic injection of incident data (e.g., location, date, and narrative) into specific XML form paths.

3. Optimization of the Data Pipeline (Microservices Integration)

The RESTful interface connecting the Node.js backend and the React frontend was optimized to handle Binary Large Object (BLOB) data streams. This enhancement ensures that generated compliance reports are delivered securely as downloadable files rather than raw text, thereby preserving document integrity for official legal submissions.

4. Current Status: Integration and Error Resolution Phase

The system is currently undergoing a rigorous testing and error-resolution phase. Current efforts focus on troubleshooting synchronization between unique database identifiers (incident_id) and the corresponding API route handlers. This intermediate phase is essential to ensure the reliability of the automated export functionality prior to final deployment.

2.1 Technical details of tasks performed

Component	Task	Technical Details	Impact
XFA Data Injection Architecture	Implement mapping for XFA-based government PDF forms	Standard PDF libraries cannot properly read XFA (XML Forms Architecture) documents used by government agencies such as WorkSafeBC. A manual field-mapping strategy was implemented using pdf-lib, addressing form fields through absolute XML paths (e.g., topmostSubform[0].Page1[0]). Document restrictions were handled by loading the PDF with the { ignoreEncryption: true } option in PDFDocument.load().	Enabled programmatic population of complex government-issued EIIR forms, allowing incident data to be automatically injected into official regulatory templates.
Binary Stream and BLOB Management	Handle PDF generation and secure file download	The frontend Axios client was configured with responseType: 'blob' to correctly process binary PDF streams returned by the server. A client-side virtual download link was implemented using window.URL.createObjectURL() to convert the binary response into a downloadable file within the browser without permanently storing the file on the server.	Ensures secure and efficient delivery of generated compliance reports while preserving document integrity and minimizing server storage requirements.
API Route & SQL Query Synchronization	Refactor API routes and optimize database queries	The Node.js REST API was refactored to support dynamic routing using /api/reports/generate/:id. Backend logic was updated to use incident_id as the primary identifier. MySQL query performance was improved through the use of a connection pool, allowing the system to retrieve only the relevant incident record required for each report generation request.	Improves system efficiency, ensures accurate data retrieval, and supports scalable report generation within the microservices architecture.
Current Technical Impediment	Debug API request–response mismatch	The system is currently addressing a 404 route mismatch error where the frontend GET request does not properly connect with the backend route handler. Debugging is being conducted using Postman and browser developer tools to inspect request headers, verify endpoint URLs, and ensure exact alignment with Express.js route definitions.	Resolving this issue will restore successful API communication, enabling reliable automated report generation and download functionality.

Screenshots are added below for further information.

localhost:3000/export

localhost:3000 says
Failed to generate PDF. Ensure the backend server is running and the template exists.

DASHBOARDANALYTICSWORKSAFEBCLogout

Compliance Automation: WorkSafeBC Export

Automated data-mapping for official EIIR (Employer Incident Investigation Report) templates

Incident ID	Incident Type	Date Reported	Status	Action
#2025060053	Slip/Fall	2026-03-14	Open	Generating...
#2025060052	Chemical Exposure	2026-02-27	Open	Generate PDF
#2025060050	Slip/Fall	2026-02-27	Open	Generate PDF
#2025060051	Slip/Fall	2026-02-05	Open	Generate PDF
#2025060049	Multiple surface and flesh wounds	2025-06-24	Reported	Generate PDF
#2025065981	Fractures	2025-06-23	Reported	Generate PDF
#2025060008	Fractures	2025-06-23	Reported	Generate PDF
#2025065835	Fractures	2025-06-19	Reported	Generate PDF
#2025065578	Effects of poison, toxic, or allergenic exposure unspecified	2025-06-11	Reported	Generate PDF
#2025055149	Fractures	2025-05-31	Reported	Generate PDF
#2025055120	Amputations, avulsions, enucleations unspecified	2025-05-30	Reported	Generate PDF
#2025055045	Intracranial injuries unspecified	2025-05-28	Reported	Generate PDF
#2025054712	Fractures	2025-05-19	Reported	Generate PDF
#2025054429	Amputations involving bone loss	2025-05-10	Reported	Generate PDF
#2025054343	Fractures	2025-05-08	Reported	Generate PDF
#2025032639	Amputations, avulsions, enucleations unspecified	2025-03-19	Reported	Generate PDF
#2025032491	Fractures	2025-03-16	Reported	Generate PDF

localhost:5000/api/reports/generate/13

localhost:5000/api/reports/generate/13

React App

localhost:5000/api/reports/generate/13

Pretty-print

```
{ "message": "Database error" }
```

3. Methodology

The project has advanced from Phase 2 (Development) into Phase 3 (Intelligence Integration), moving the SafeSight system from a "System of Record" to a "System of Insight." This transition was executed using a specialized, multi-layered methodology focused on data liquidity and automated regulatory mapping.

3.1 Issues encountered and how they were resolved

The transition into Phase 3 (Intelligence Integration) introduced several technical challenges related to data synchronization and document automation. While most issues have been successfully resolved, a critical integration bottleneck is currently under investigation.

1. Schema Identifier Mismatch (Resolved)

Issue: During the implementation of the PDF export feature, the system was unable to retrieve specific incident records. Initial diagnostics revealed a mismatch between the frontend's expected id property and the MySQL database's primary key, incident_id.

Resolution: A full-stack refactor was performed on the reportController.js SQL queries and the React WorkSafeBCExport mapping logic to ensure strict alignment with the incident_id schema.

2. XFA PDF Form Incompatibility (Resolved)

Issue: The official WorkSafeBC EIIR template is built using XFA (XML Forms Architecture), which most standard PDF libraries cannot natively read or modify. Initial attempts to populate the form resulted in blank documents.

Resolution: A manual mapping methodology was developed using absolute XML paths (e.g., topmostSubform[0]). In addition, an encryption-bypass configuration was implemented using the ignoreEncryption: true flag within the pdf-lib library to enable controlled form manipulation.

3. Binary Data Corruption (Resolved)

Issue: Early versions of the download functionality produced "Failed to Load PDF" errors in the browser because the generated PDF was transmitted as JSON text, resulting in corrupted binary data.

Resolution: The Axios HTTP client was reconfigured with responseType: 'blob', and a client-side Blob-to-URL conversion mechanism was implemented to preserve binary integrity during file transfer.

4. Current Unresolved Issue: API Route Handshake Failure (Ongoing)

Status: Critical / Unresolved

Description: A **404 Not Found** error is occurring during the final integration of the “Generate PDF” functionality. Although backend routes for /api/incidents and /api/reports/generate/:id are correctly defined, the frontend request intermittently fails to establish a successful connection with the server.

Impact: This issue prevents end users from triggering automated PDF generation directly through the user interface, despite the database queries and PDF generation logic functioning correctly in isolated testing environments.

Current Action Plan: A comprehensive network-level inspection is currently underway. **Browser Developer Tools** are being used to analyze request headers and URL construction patterns. The investigation is focused on determining whether the root cause lies in **Express.js route ordering** or a **CORS (Cross-Origin Resource Sharing) preflight conflict**.

3.2 Changes made to the original project proposal

A strategic pivot was made regarding the specific regulatory document targeted for automation. This change reflects a deeper understanding of the technical constraints of PDF architectures and the operational needs of the hospitality safety sector.

1. Transition from Form 7 to EIIR (Employer Incident Investigation Report)

- **Original Plan:** The initial proposal aimed to automate the WorkSafeBC Form 7 (Employer’s Report of Injury or Occupational Disease).
- **The Change:** The project has been redirected to automate the WorkSafeBC EIIR (Form 52E40).

Reasoning (Technical & Operational):

- **Data Availability:** Form 7 requires highly sensitive personal information (Social Insurance Numbers, specific medical history) that typically resides in protected HR/Payroll silos rather than a floor-level Safety Management System.
- **Incident Lifecycle:** The EIIR is the more immediate "System of Insight" document. It focuses on the investigation of the incident (root causes, work area, immediate actions), which aligns perfectly with the predictive data being generated by the SafeSight AI engine.
- **PDF Architecture:** The EIIR template allowed for a more robust implementation of the XFA (XML Forms Architecture) mapping methodology developed during this phase, providing a more reliable "Proof of Concept" for compliance automation.

3.4 Updated timeline

Phase 3: Intelligence & Predictive Analytics (Completed/Refining)

- Integrated the NLP engine to sanitize incident descriptions.
- Deployed a Random Forest risk prediction model to identify departmental risk hotspots.
- Status: Backend functionality is operational; the *Prescriptive Actions* user interface is currently being refined.

Phase 4: Compliance Automation & Document Exporting (Active Phase)

- Developed an XFA mapping engine to automatically populate government-issued templates.
- Implemented a Binary Stream and Blob API for secure PDF downloads without server-side file storage.
- Status: Currently resolving an API handshake (404) error to complete the end-to-end reporting workflow.

3.5 Next Steps

Phase 4: Compliance Automation & Final Enhancements

- **API Integration Fix:** Resolve the 404 handshake error by refactoring Express.js routes and reviewing CORS configuration to ensure proper communication between the React frontend and Node.js backend.
- **XFA Form Completion:** Finalize mapping for the WorkSafeBC EIIR (Form 52E40), including narrative fields and automated checkbox logic based on incident severity.
- **AI Narrative Enhancement:** Introduce an NLP layer to refine incident descriptions into professional language suitable for regulatory reporting.
- **Frontend UX Improvements:** Add loading notifications and improve dashboard integration to provide clear feedback and a streamlined export interface for users.

4 Repository Check-In Summary

Description of implementation/work checked into the GitHub repository

The GitHub repository has been updated to reflect the transition from a monolithic script to a Modular Full-Stack Application. The current check-in includes the core API infrastructure, the AI-to-Backend bridge, and the newly developed Compliance Automation engine. Key updates in this commit include:

- **Database Integration:** Implementation of the relational schema using `incident_id` as the primary key.
- **XFA Mapping Logic:** Addition of the `reportController.js` which handles the complex XML-path injection for official WorkSafeBC PDF forms.
- **Client-Side Blob Handling:** Deployment of the React frontend logic required to process binary streams into downloadable reports.

Folder/file structure used (Implementation / DocumentsAndReports)

The repository follows a strict separation of concerns to ensure scalability and ease of maintenance. The structure is divided into two primary top-level directories:

Implementation/

AI_Service/

- **predict.py:** Core Machine Learning engine (Random Forest) and Flask API.
- **requirements.txt:** Documentation of Python dependencies.
- **venv/:** Isolated virtual environment for AI dependency management.

BackEnd/

- **server.js:** Updated with the `/api/reports/generate/:id` route and enhanced CORS configurations to support binary data transfers.
- **controllers/reportController.js (New):** Contains the pdf-lib logic for XFA field mapping and SQL data extraction using `incident_id`.

- **assets/ (New Directory):** Stores the official WorkSafeBC EIIR PDF templates used as the base for automation.

FrontEnd/

- **src/components/:** Added WorkSafeBCExport.js to provide the UI for generating and downloading reports. Updated existing components to use incident_id for data consistency.
- **App.js:** Reconfigured to include the Compliance Automation route and ensure the "Export" feature is restricted to authorized roles.

DocumentsAndReports/

- **Progress Report 3 (new):** Detailed documentation of the "System of Automation" implementation.
- **Work Log (updated)**
- **AI Usage Documentation (updated)**

Commits

Number of commits so far: 23

Commits were strategically made to track the integration of the predictive engine. Major commit milestones include:

- feat: implement XFA PDF mapping for EIIR

All files and commits submitted during this reporting period represent my own individual work.

Work Log

The following work log reflects activities completed since the last progress report 2 reporting period. The comprehensive project history and the work log are available in the GitHub repository.

Date	Number of Hours	Description	Remarks
March 3, 2026	3	Refining Random Forest Model and Shift/Department hotspot mapping.	AI Service testing completed.
March 4, 2026	2	Researching WorkSafeBC EIIR (Form 52E40) PDF structure and XFA requirements.	Decided to pivot from Form 7.
March 5, 2026	2	Implementing pdf-lib in the backend and creating XFA field mapping logic.	Established reportController.js.
March 6, 2026	2	Developing Frontend "Export" component and handling binary BLOB data streams.	PDF download button implemented.
March 8, 2026	2	Troubleshooting API Handshake and resolving "Database Error" in report generation.	In Progress: Debugging 500 error shown in logs.
March 9, 2026	2	Progress Report 3 Preparation	

AI Use Disclosure

The following section outlines AI-related activities completed since the last midterm reporting period. Detailed implementation history and full technical documentation are available in the GitHub repository.

AI Tool Name	Version / Account Type	Specific Feature Used	Prompt Used (Summary)	Value Addition (Student Contribution)
Gemini	3 Flash / Free Tier	Code Analysis & Documentation	"How do I programmatically fill a WorkSafeBC EIIR PDF form using JavaScript?"	Prevented hours of wasted effort by identifying early that standard getFields() methods would fail, requiring a "Deep Mapping" approach.
Gemini	3 Flash / Free Tier	Debugging / Log Interpretation	"Why does pdf-lib return 'Invalid Object Reference' when loading an encrypted government PDF?"	Engineered a "Zero-Storage" flow, ensuring sensitive data exists only in temporary memory, enhancing user data privacy.
Gemini	3 Flash / Free Tier	Architecture Design	"How to download a PDF from a Node.js backend to a React frontend without saving it to disk?"	Troubleshooting corrupted PDF downloads and binary data handling.
Gemini	3 Flash / Free Tier	Code Refactoring	"Update my Express.js SQL query to use 'incident_id' instead of 'id' and sync with the frontend."	Guaranteed 100% accuracy in data-to-field mapping, ensuring the correct incident record is always injected into the legal export.
Gemini	3 Flash / Free Tier	Systematic Debugging	"My GET request to /api/reports/generate/13 is returning a 404. Check my server.js route order."	Provided a systematic framework to isolate failures between the Express router and the MySQL connection pool.

ChatGPT	GPT-5.3 / Free Plan	Concept clarification and debugging guidance	"Prompts describing API routing errors, PDF generation issues, and data flow between the React frontend, Node.js backend, and MySQL database"	Applied the conceptual guidance to troubleshoot system integration issues, tested solutions using developer tools and Postman, and implemented the final working logic in the application
GitHub Copilot	Student / IDE Integration	Code suggestion and auto-completion	"Context-based code suggestions for JavaScript, Express.js routing, and React component logic"	Reviewed, modified, and optimized suggested code to ensure compatibility with the microservices architecture and database schema
Python NLP Libraries (via AI assistance)	Local Development Environment	Natural Language Processing preparation	"Prompts related to sanitizing and improving incident narrative descriptions for professional reporting"	Designed how the NLP layer integrates with the incident reporting pipeline and adapted outputs to match regulatory documentation standards
ChatGPT	GPT-5.3 / Free Plan	Technical writing and documentation refinement	"Prompts requesting restructuring of technical descriptions, summaries, and professional wording for project documentation and reports"	Designed how the NLP layer integrates with the incident reporting pipeline and adapted outputs to match regulatory documentation standards